FORMATION « PYTHON EN 24 HEURES » | SÉANCE 12

Débogage et synthèse finale

Chapitre 21 et annexe du Tome 1

1 h de cours (présentation Beamer) • 1 h de pratique (notebook Jupyter)

HOUNKPE Mawulikplimi Roland et DJABON Ounimborbitibou

Objectifs de la séance

- Appliquer la méthode de débogage en quatre temps
- Instrumenter un code avec print({x=}) et assert
- Connaître les commandes de base de breakpoint() / pdb
- Reconnaître les six pièges classiques du débutant
- Faire le bilan des douze séances et se préparer au Tome 2

Rappel de la séance 11

Dernière séance : nous consolidons tout le parcours, des premières conditions aux exceptions, et nous apprenons à chasser la plus sournoise des erreurs : l'erreur logique.

- Trois familles : syntaxe, exécution, logique
- Traceback : se lit de bas en haut, la dernière ligne nomme l'exception
- try, except, else, finally : le protocole complet
- raise et exceptions personnalisées : nommer les règles métier
- EAFP : agir puis réparer, l'idiome Python

La méthode en quatre temps

1. Reproduire

- Rendre l'erreur régulière
- Trouver le plus petit jeu de données qui plante
- Une erreur qu'on ne reproduit pas ne se corrige pas

2. Localiser

- Traceback pour les plantages
- print({x=}) pour les faux résultats
- Diviser le code en zones de doute

3. Corriger

- La plus petite modification possible
- Un bug corrigé, une fois
- Jamais trois correctifs en même temps

4. Vérifier

- Le cas qui plantait passe
- Les autres cas passent encore
- Un assert protège l'avenir

print instrumenté : {x=}

```python
def moyenne_ponderee(notes, coeffs):
    total = 0
    poids = 0
    for note, coef in zip(notes, coeffs):
        total += note * coef
        poids += coef
    print(f"{notes=} {coeffs=} {total=} {poids=}")
    return total / poids

moyenne_ponderee([12, 15], [1, 2])
```

Le f-string {x=} affiche le nom ET la valeur : x=12. Un print de plus en haut, un en bas, et la zone de doute se réduit en deux exécutions.

assert : un contrôle qui parle

```
def normaliser(note):
    if note < 0:
        note = 0
    return note

assert normaliser(-5) == 0, "note négative ramenée à zéro"
assert normaliser(15) == 15, "note correcte inchangée"
print("Les deux règles tiennent")
```

assert(condition, message) : si la condition est fausse, AssertionError avec le message. Pour VERIFIER, pas pour VALIDER : la séance 11 traitait la validation.

breakpoint() : le microscope

- Insérez breakpoint() là où le doute vit
- L'exécution se fige et ouvre pdb en console
- Commandes clés : p variable, n (suivant), s (entrer), c (continuer), q (quitter)
- Vous inspectez l'état réel, pas une supposition
- Dans un notebook, préférez print ; pdb brille dans un script

```python
def calcul_mystere(valeurs):
    total = 0
    for v in valeurs:
        breakpoint()   # l'exécution se fige ici
        total += v
    return total
# p valeurs  -> la liste
# p total    -> l'accumulateur
# n          -> tour suivant
# c          -> reprendre
```

pdb dans le terminal

```
$ python -m pdb bilan.py
> bilan.py(1)<module>()
(Pdb) b 12            # point d'arrêt ligne 12
(Pdb) c               # continuer jusqu'au point
> bilan.py(12)rapport_ventes()
-> ca = chiffre_affaires(ventes_produit)
(Pdb) p ventes_produit
[(1200, 3), (1200, 1)]
(Pdb) n               # ligne suivante
(Pdb) q               # quitter
```

La même session que breakpoint(), mais lancée depuis le shell : python -m pdb suivi du nom du script. Les commandes sont identiques.

Les six pièges classiques

Piège	Symptôme	Parade
= au lieu de ==	SyntaxError ou affectation surprise	relire la ligne du traceback
Borne de fin de range	un tour de trop ou de moins	tester avec n = 1
Liste partagée	modifications qui se propagent	copier avec .copy()
Mutable en défaut	paramètre qui accumule entre appels	défaut None puis création
except nu	les bugs disparaissent en silence	attraper un type précis
Indentation trompeuse	une ligne dans le mauvais bloc	afficher les espaces

Imprimez ce tableau : la moitié des bugs de première année y figure déjà.

Écrire du code facile à déboguer

- Des fonctions **courtes** qui font une seule chose
- Des noms qui disent l'intention : ‘total_valide‘ plutôt que ‘tv‘
- Un point d'entrée protégé : les validations à la frontière (séance 11)
- Des asserts pour les invariants : ce qui doit toujours être vrai
- Tester au fil de l'eau plutôt qu'à la fin : chaque séance du TP le démontre

Le parcours en un coup d'oeil

Fondations

- S01 et S02 : algorithmique, conditions, boucles
- S03 et S04 : types, conditions et boucles Python

Données et fonctions

- S05 et S06 : listes, tuples, compréhensions, dict, set
- S07 : closures, décorateurs, chaînes, fichiers

Objets et robustesse

- S08 et S09 : classes, encapsulation, héritage, dunder
- S10 et S11 : projet bancaire, exceptions

Et après ? Le Tome 2

- **Modules et paquets** : organiser un projet en plusieurs fichiers
- **Tests automatisés** : unittest et pytest, la ceinture de sécurité du code
- **Expressions régulières** : la validation de texte au niveau supérieur
- **Bases de données** : sqlite3 et la persistance réelle
- **Algorithmes avancés** : récursivité, complexité, structures de données
- Votre formation de 24 h se poursuit : chaque notion du Tome 1 y est prolongée

Astuce

Le journal de bugs

Tenez un journal de vos trois derniers bugs : symptôme, cause, correction. Au bout de dix entrées, vos motifs personnels apparaissent : la prévention devient personnelle, et le débogage un réflexe plutôt qu'une corvée.

À retenir

Le bilan du Tome 1

Vous savez décrire un algorithme, le traduire en Python, organiser les données, écrire des fonctions et des closures, construire des classes qui se parlent et rendre le tout robuste face aux erreurs. La séance 12 ajoute la compétence qui fait la différence en pratique : retrouver et corriger un bug avec méthode. Le Tome 2 vous attend avec les modules, les tests automatisés, les expressions régulières et les bases de données.

synthèse

TP spécial de fin de formation : deux exercices de débogage, un quiz auto corrigé couvrant les douze séances, et un programme mystère à cinq bugs. Pour les exercices 1 et 3, le squelette CONTIENT le bug : exécutez, observez, corrigez.

- **Exercice 1 (Basique)** : Chasse au bug : la moyenne qui ment
- **Exercice 2 (Basique)** : Écrire ses propres tests
- **Exercice 3 (Intermédiaire)** : Chasse au bug : le décalage de un
- **Exercice 4 (Intermédiaire)** : Quiz de synthèse auto corrigé
- **Exercice 5 (Défi)** : Le programme mystère : cinq bugs

Ouvrez le notebook « Seance_12_debogage_et_synthese.ipynb » pour commencer.

Pour la prochaine séance

Conseil de travail

Formation terminée : 24 heures de travail, 12 séances, 60 exercices corrigés. Le Tome 2 du cours prend le relais avec les modules, les tests automatisés, les expressions régulières et les bases de données. À très bientôt !